The Mathematics of Deception

Can Mathematics Design an Optimal Strategy for Detecting a Lie?

1. Introduction

A painting disappears from a museum overnight. The night watchman says: *"I was at my post all night. I never left the east wing."*

He is either telling the truth or he isn't. That fact is already fixed; it happened last night. The trouble is that we cannot see it. All we can see is what he says, how he says it, and how his answers fit together.

This is the situation this project is about: a hidden truth, and a stream of imperfect clues about it. Detectives, doctors, scientists and courts all face versions of the same problem. You never observe the thing you care about directly. You observe evidence, and you have to decide how much that evidence should change your mind.

Two questions follow naturally. First: when a new piece of evidence arrives, **by exactly how much** should our belief change? Second: if we are allowed to ask only a few questions, **which question should we ask next**? The first is a question about updating beliefs. The second is a question about choosing well under uncertainty. Mathematics has surprisingly precise answers to both, but only inside a carefully defined model.

That last phrase matters, so it's worth being blunt about it from the start. This project does not build a lie detector for real people. It builds a **mathematical model** of a questioning process: a small invented world with explicit rules. Throughout the project we keep four things separate:

Thing	What it is	Can we reason about it exactly?
The mathematical model	The rules we write down: hypotheses, probabilities, how answers are generated	Yes, completely
The assumptions	The choices we made to build the model, each one a simplification	Yes, but they can be wrong
A real human being	Someone with memory, nerves, motives and misunderstandings	No
Psychological lie detection	An empirical science studying real behaviour	Not with this project's tools

Inside the model, every conclusion can be proved. Whether the model resembles real people is a separate question, and Section 15 comes back to it honestly.

The path of the investigation is simple to state. We start not knowing the truth. We collect evidence. Evidence changes belief. Some questions turn out to be far more informative than others, and

information theory lets us measure how much. With that measurement, mathematics can choose the next question for us. We turn the whole thing into an algorithm, and then we test whether it actually works.

2. Research Question

The question this project answers is:

> ***Given uncertainty about whether a statement is truthful, can Bayesian inference and information theory be combined to design a sequence of questions that maximises the probability of identifying the truth with the fewest questions?***

It splits into three smaller questions, each one building on the last:

1. **Updating.** When we observe an answer, how should the probability that the statement is a lie change? (Sections 4 to 6)
2. **Measuring.** How much uncertainty do we have, and how much does a given question reduce it on average? (Sections 7 to 10)
3. **Choosing.** If we always pick the question that reduces uncertainty the most, do we reach the right conclusion faster than by picking questions at random? (Sections 11 to 14)

The third question is the one that can be tested. It makes a concrete, checkable prediction, and Section 12 checks it by simulation.

3. Building the Model

Before any calculation, we have to say exactly what world we are working in. A model is only as honest as the list of assumptions behind it, so each assumption below comes with its reason.

The hidden variable. The watchman's statement is either a lie or the truth. We write this as a variable L:

$$L = \begin{cases} 1 & \text{the statement is a lie} \\ 0 & \text{the statement is truthful} \end{cases}$$

We write T (for "truthful") as shorthand for L = 0, and ¬L ("not L") means the same thing.

The prior. Before any questioning, we must say how likely a lie is. With no information either way, we start with

$$P(L) = 0.5, \qquad P(T) = 0.5$$

This choice is not a fact about the watchman. It expresses total indifference, and later (Section 14) we test what happens when the starting belief is different.

Evidence. Each question produces an answer, and each answer has some observable feature: a hesitation, a detail that contradicts an earlier answer, an unusually precise time. We call an observed feature E. The model's key idea is that liars and truth-tellers produce E with *different* probabilities, written P(E | L) and P(E | T).

The assumptions of the baseline model:

Assumption	Required by the mathematics, or a modelling choice?	Why we make it
A1. Exactly two hypotheses: Lie or Truth	Modelling choice. Bayes' theorem works for any number of hypotheses.	The simplest non-trivial uncertainty; more possibilities come in Section 10
A2. P(E | L) and P(E | T) are known exactly for every question	Bayes' theorem needs *some* values for them. Knowing the *true* values is a modelling choice.	Lets us study ideal reasoning first; Section 14B drops it
A3. Answers are imperfect: 0 < P(E | L), P(E | T) < 1	Modelling choice. If a likelihood were 0, Bayes still works; the evidence just becomes conclusive.	Perfect tells would make the problem trivial and unrealistic
A4. Answers are conditionally independent given the truth	Modelling choice about the world, but mathematically required for the multiplication rule in Section 6	Makes evidence combine simply; Section 14C drops it
A5. The prior is fixed before questioning, with 0 < P(L) < 1	The value is a choice. Keeping it strictly between 0 and 1 is required: a prior of exactly 0 or 1 can never be changed by any evidence.	Section 14A tests other starting beliefs
A6. The suspect does not adapt to our questions	Modelling choice	Removes strategy; the adversarial liar comes in Section 14E
A7. Each question has a yes/no-type answer and a limited number of questions can be asked	Modelling choice	Keeps calculations small and creates the need to choose; Section 14D varies the budget

The baseline model is deliberately too simple. That is a strength rather than a flaw: once we fully understand it, each assumption can be removed on its own, and we can see exactly what that assumption was doing.

4. Probability and Conditional Probability

What does P(L) = 0.5 mean? The watchman either lied or he didn't. So the 0.5 cannot be a property of the event itself. It describes *our uncertainty* about it. This is the **Bayesian interpretation**: probability as a degree of belief, which gets updated as information arrives. The older **frequentist interpretation** treats probability as a long-run frequency, such as a fair coin landing heads half the time.

The two views fit together through a useful trick. Imagine many cases exactly like ours: 100 watchmen in 100 museums. Then "a degree of belief of 0.3" becomes "30 out of 100". Counting imaginary people is much harder to get wrong than juggling abstract formulas, so this project uses it constantly.

A population of statements. Suppose that in our model world, 100 statements are made. One observable feature is whether the speaker hesitates before answering:

	Hesitates	Doesn't hesitate	Total
Lie	18	12	30
Truth	14	56	70
Total	32	68	100

The probability that a statement is a lie is 30/100 = 0.3.

Conditional probability: shrinking the universe. Now ask: *given* that a person is lying, what is the probability they hesitate? The word "given" tells us to throw away everyone who isn't lying. The universe shrinks from 100 statements to the 30 lies, and 18 of those hesitate:

$$P(\text{Hesitate} \mid L) = \frac{18}{30} = 0.6$$

The vertical bar is read "given". Whatever is written to its right is what we already know, and we restrict attention to that part of the world.

To turn this into a general rule, rewrite the fraction as proportions of the whole population:

$$\frac{18}{30} = \frac{18/100}{30/100} = \frac{P(\text{Hesitate and } L)}{P(L)}$$

This gives the definition of conditional probability:

$$P(A \mid B) = \frac{P(A \text{ and } B)}{P(B)}, \qquad P(B) > 0$$

It is not a new law to memorise. It is the shrinking-universe idea written as fractions. Rearranging it gives the **multiplication rule**, which we will use constantly:

$$P(A \text{ and } B) = P(A \mid B)\, P(B)$$

In words: the chance of both happening equals the chance of B, times the chance of A once B is known to have happened.

The trap. Read the table the other way. Given that someone hesitates, what is the probability they are lying? Now the universe is the 32 hesitators, and 18 of them lied:

$$P(L \mid \text{Hesitate}) = \frac{18}{32} = 0.5625$$

The two numbers, 0.6 and 0.5625, answer completely different questions. P(Hesitate | L) describes how liars behave. P(L | Hesitate) describes how likely *this* person is to be lying, given what we saw. Confusing them is a famous error, known as the **prosecutor's fallacy**.

This difference is the engine of the whole project. Our model naturally tells us numbers of the first kind: how liars and truth-tellers behave. The investigator needs a number of the second kind. With a full table the conversion is easy, because we just count. Without a table, we need a formula that turns P(E | L) into P(L | E). That formula is Bayes' theorem.

5. Bayes' Theorem

Deriving it. Bayes' theorem comes from writing the same quantity in two ways. "L and E" can be split using the multiplication rule in either order:

$$P(L \text{ and } E) = P(L \mid E)\,P(E) \qquad \text{and} \qquad P(L \text{ and } E) = P(E \mid L)\,P(L)$$

Both equal the same thing, so they equal each other. Dividing by P(E) gives:

$$P(L \mid E) = \frac{P(E \mid L)\,P(L)}{P(E)}$$

One piece is still missing: P(E), the overall chance of seeing this evidence. Every statement is either a lie or the truth, so E can happen in exactly two ways: "E and L" or "E and T". Adding them, and using the multiplication rule on each, gives the **law of total probability**:

$$P(E) = P(E \mid L)\,P(L) + P(E \mid T)\,P(T)$$

Substituting this in gives Bayes' theorem in the form used for the rest of the project:

$$P(L \mid E) = \frac{P(E \mid L)\,P(L)}{P(E \mid L)\,P(L) + P(E \mid T)\,P(T)}$$

What each part means.

Symbol	Name	Meaning in the investigation
P(L)	Prior	How likely a lie seemed *before* this evidence
P(E | L)	Likelihood (under Lie)	How often a liar would produce this evidence
P(E | T)	Likelihood (under Truth)	How often a truth-teller would produce it
P(E)	Evidence (total probability)	How often we would see this evidence at all
P(L | E)	Posterior	How likely a lie is *after* seeing the evidence

The structure is easy to read. The numerator is "the ways E can happen through a lie". The denominator is "all the ways E can happen". The posterior is simply the lie's share of the total, which is exactly the shrinking-universe idea from Section 4, without needing a table.

Check against the table. With P(L) = 0.3, P(Hesitate | L) = 0.6 and P(Hesitate | T) = 0.2:

$$P(L \mid \text{Hesitate}) = \frac{0.6 \times 0.3}{0.6 \times 0.3 + 0.2 \times 0.7} = \frac{0.18}{0.32} = 0.5625$$

This matches the 18/32 we counted directly. The formula is doing the counting for us.

The first experiment. Back to the watchman. We start with P(L) = 0.5 and ask: *"What time did you do your last round?"* He gives a consistent, easily checked answer. Call this evidence E. In our model:

$$P(E \mid L) = 0.4, \qquad P(E \mid T) = 0.8$$

That is, a truth-teller gives a consistent answer 80% of the time and a liar only 40% of the time. Then:

$$P(L \mid E) = \frac{0.4 \times 0.5}{0.4 \times 0.5 + 0.8 \times 0.5} = \frac{0.2}{0.6} = \frac{1}{3} \approx 0.333$$

One consistent answer moves the probability of a lie from 0.5 down to about 0.333. It does not prove he is honest; liars give consistent answers 40% of the time. But truthful people do so twice as often, and that ratio is what shifts our belief.

The deeper point. Here the prior cancelled out of the calculation. That happened because P(L) = P(T) = 0.5, so both terms carried the same factor of 0.5; with an unequal prior it would not cancel. The general lesson is that what the evidence contributes depends only on the ratio P(E | L) : P(E | T) = 0.4 : 0.8, not on either number alone. Section 6 makes this precise.

6. Evidence Accumulation

Yesterday's posterior is today's prior. After one answer, P(L | E_1) is our new state of belief. When a second answer arrives, we apply Bayes' theorem again, using that posterior as the new prior. Belief is never finished. It is just the current step in a chain of updates.

A cleaner tool: odds. Repeating the full formula quickly gets messy, so we rewrite Bayes' theorem in a simpler shape. Write Bayes' theorem for L, then write the same thing for T:

$$P(L \mid E) = \frac{P(E \mid L)\,P(L)}{P(E)}, \qquad P(T \mid E) = \frac{P(E \mid T)\,P(T)}{P(E)}$$

Divide the first by the second. The awkward P(E) cancels:

$$\underbrace{\frac{P(L \mid E)}{P(T \mid E)}}_{\text{posterior odds}} = \underbrace{\frac{P(E \mid L)}{P(E \mid T)}}_{\text{likelihood ratio}} \times \underbrace{\frac{P(L)}{P(T)}}_{\text{prior odds}}$$

The **odds** of a lie are P(L)/P(T); odds of 3 mean "3 to 1 in favour of a lie". To convert back, use P(L) = odds / (1 + odds). The **likelihood ratio** (LR) measures how strongly a piece of evidence points one way: LR > 1

points towards a lie, LR < 1 towards the truth, and LR = 1 means the evidence is useless. This one equation says precisely what Section 5 hinted at: the strength of evidence is the ratio.

Several pieces of evidence. If answers are conditionally independent given the truth (assumption A4), their likelihoods multiply, so the updates simply chain:

$$\text{posterior odds} = \text{LR}_1 \times \text{LR}_2 \times \cdots \times \text{LR}_n \times \text{prior odds}$$

The investigation continues. The watchman is asked four questions. The model's probabilities for each observed feature, and the resulting belief, are below.

Step	Evidence observed	P(E | L)	P(E | T)	LR	Odds of lie	P(L)
Start	none				1	0.500
1	Consistent answer about last round	0.4	0.8	0.5	0.5	0.333
2	Hesitates when asked about the alarm	0.6	0.2	3	1.5	0.600
3	Contradicts his earlier timing	0.5	0.1	5	7.5	0.882
4	Consistent answer about the keys	0.4	0.8	0.5	3.75	0.789

Belief moves gradually. No single answer settles the matter, and it swings in both directions: down, then up, sharply up, then partly back. The contradiction in step 3 did the most work because its likelihood ratio, 5, was the largest.

Two consequences follow directly from the multiplication. First, **order doesn't matter**: multiplying 0.5 × 3 × 5 × 0.5 in any order gives 3.75, so updating one answer at a time reaches the same belief as updating on all four at once. Second, **evidence adds on a log scale**. Taking logarithms turns the product into a sum:

$$\log(\text{posterior odds}) = \log(\text{prior odds}) + \sum_{i=1}^{n} \log\left(\text{LR}_i\right)$$

Each answer contributes an amount of "weight of evidence", pushing positive towards a lie or negative towards the truth. Ideas closely related to likelihood ratios and accumulating statistical evidence also played an important role in wartime codebreaking.

Where this rests on an assumption. Multiplying likelihoods is only valid if the answers are conditionally independent given the truth. If the watchman hesitates on one question because he's nervous in general, his hesitation on the next question is not fresh evidence, and multiplying would count the same clue twice. Section 14C explores what happens when this assumption breaks.

This leaves an obvious question. Step 3 moved our belief far more than step 1. Could we have known *in advance* which question would be more useful? That is where the project turns from updating beliefs to choosing questions.

7. Which Question Is Better?

In Section 6, the contradiction question moved our belief the most. But we only know that because we already saw the answer. Standing in front of the watchman, before asking anything, we face a different problem: **a question is a gamble over its possible answers.**

Take the contradiction question ("Does your timing contradict what you said earlier?") with P(yes | L) = 0.5 and P(yes | T) = 0.1, and suppose we start at P(L) = 0.5. Two things can happen:

Answer	How likely?	Belief afterwards
Yes, a contradiction	P(yes) = 0.5 × 0.5 + 0.1 × 0.5 = 0.30	P(L | yes) = 0.25 / 0.30 ≈ 0.833
No contradiction	P(no) = 0.70	P(L | no) = 0.25 / 0.70 ≈ 0.357

The "how likely" column is the law of total probability from Section 5. So the question *might* push us most of the way to certainty, but usually it won't: 70% of the time we get a mild nudge towards innocence.

A tempting idea that fails. Why not judge a question by how far it moves our belief on average? Let's calculate the average belief after the question:

$$P(\text{yes})\,P(L \mid \text{yes}) + P(\text{no})\,P(L \mid \text{no}) = 0.30 \times 0.833 + 0.70 \times 0.357 = 0.25 + 0.25 = 0.5$$

It comes back to exactly 0.5, the prior. This is not luck. By the multiplication rule, P(yes) × P(L | yes) is just P(L and yes), and similarly for "no". Adding them gives P(L and yes) + P(L and no) = P(L). So for *every* question:

$$\sum_{e} P\,(e)\,P(L \mid e) = P(L)$$

On average, a question never moves our belief at all. Every question pushes belief up with some answers and down with others, and the pushes always balance exactly. If they didn't, we could predict which way our belief was going to move, and then we should have moved it already.

So "average belief afterwards" is useless for comparing questions. What differs between questions is not where belief ends up on average, but **how spread out** the possible end-points are. A good question sends us, on average, to beliefs that are *less uncertain*.

That tells us exactly what we need:

1. a single number that measures how uncertain we are at any belief P(L) = p,
2. that number calculated after each possible answer,
3. the average of those numbers, weighted by how likely each answer is.

A sensible uncertainty measure should be 0 when we are certain (p = 0 or p = 1), largest when we are completely unsure (p = 0.5), and the same for p and 1 − p, because being 90% sure of a lie is as certain as being 90% sure of the truth. There is a measure with exactly these properties, and a very good reason for using it. It is called entropy.

8. Entropy

Start with surprise. Before measuring uncertainty, measure something simpler: how surprised you are when an event happens. If something had probability 1, it tells you nothing new, so its surprise should be 0. A fair coin landing heads is mildly surprising. Rolling a six is more surprising. The rarer the event, the bigger the surprise.

There's one more requirement. If two independent things happen, say a coin lands heads *and* a die shows six, the total surprise should be the surprise of one plus the surprise of the other. But probabilities of independent events *multiply* (1/2 × 1/6 = 1/12). We need a function that turns multiplication into addition, and that is exactly what a logarithm does: log(xy) = log x + log y. So we define the surprise of an outcome with probability q as

$$\text{surprise}(q) = \log_2\left(\frac{1}{q}\right) = -\log_2 q$$

This gives surprise(1) = 0, surprise(1/2) = 1, surprise(1/4) = 2 and surprise(1/64) = 6. An event with probability 1/4 is as surprising as two fair coins both landing heads.

Entropy is average surprise. Uncertainty is how surprised we *expect* to be when the truth is revealed. If our belief is P(L) = p, then with probability p the truth is "lie" (surprise $-\log_2 p$) and with probability 1 − p it is "truth" (surprise $-\log_2(1-p)$). Averaging gives the **binary entropy**:

$$H(p) = -p\log_2 p - (1-p)\log_2(1-p)$$

For a situation with many possible outcomes having probabilities p_1, p_2, …, the same idea gives the general form, which Section 10 will need:

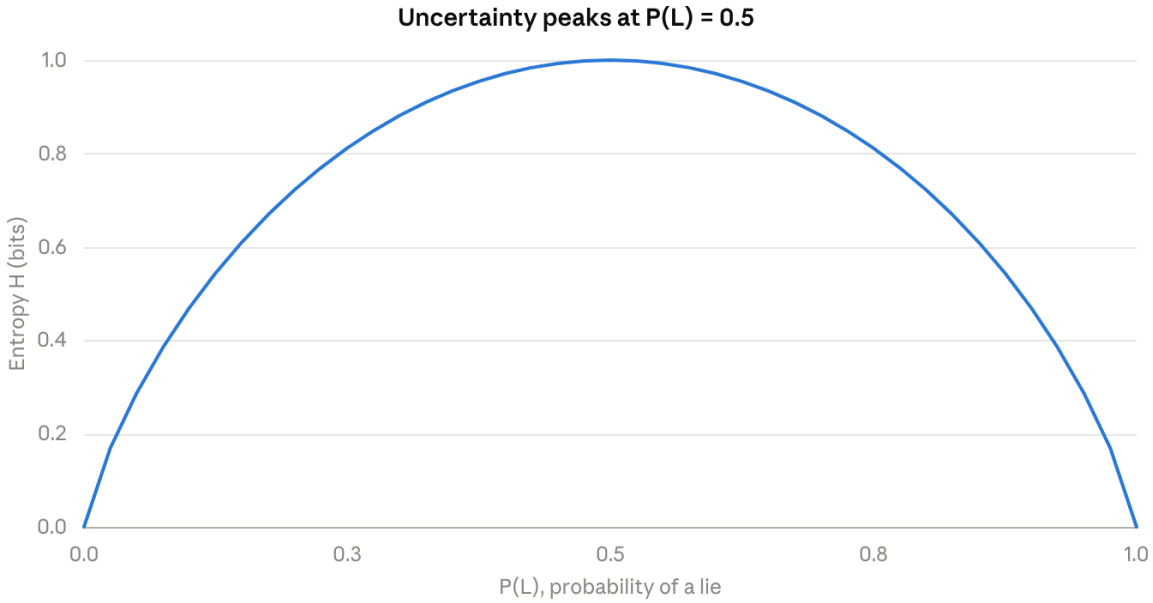
$$H = -\sum_i p_i \log_2 p_i$$

Some values.

Belief P(L)	0.5	0.6	0.7	0.8	0.9	0.99	1
Entropy H (bits)	1.000	0.971	0.881	0.722	0.469	0.081	0

At p = 0.5: H = $-0.5 \log_2 0.5 - 0.5 \log_2 0.5 = 0.5 + 0.5 =$ **1 bit**. At p = 0.9: H = $-0.9 \log_2 0.9 - 0.1 \log_2 0.1 \approx 0.137 + 0.332 = 0.469$ bits. Being 90% sure still leaves almost half a bit of uncertainty, which is more than intuition suggests.

Why H(1) = 0. At p = 1 the first term is $-1 \times \log_2 1 = 0$. The second term is $0 \times \log_2 0$, where $\log_2 0$ is undefined. But as p gets very small, $p \log_2 p$ gets closer and closer to 0 (the p shrinks faster than the log grows), so we define $0 \times \log_2 0 = 0$. Certainty means no expected surprise.



Binary entropy H(p) in bits. Zero at certainty, 1 bit at a 50/50 belief, symmetric about 0.5.

computed from H(p) · p from 0 to 1 in steps of 0.025

Why the maximum is at p = 0.5. The graph shows it, and calculus proves it. Using the derivative of $p \log_2 p$, which is $\log_2 p + 1/\ln 2$:

$$H'(p) = -\log_2 p - \frac{1}{\ln 2} + \log_2(1-p) + \frac{1}{\ln 2} = \log_2\left(\frac{1-p}{p}\right)$$

This is positive when p < 0.5 (because $(1 - p)/p > 1$), zero at p = 0.5, and negative when p > 0.5. So H rises until p = 0.5 and falls after it. It is also symmetric, H(p) = H(1 − p), because swapping p and 1 − p just swaps the two terms.

The curve is **concave**: it bulges upwards like a dome. That shape will turn out to be the reason questions always help on average (Section 9).

Why "bit" is the right unit. One bit is the uncertainty of a fair coin flip, and it is also exactly what one perfect yes/no answer removes. At P(L) = 0.5, a single perfectly reliable question ("Did you lie?" answered by a truth serum) would take us from 1 bit to 0 bits. In general, H counts how many perfect

yes/no questions the uncertainty is worth. That's why computers measure information in bits: a bit is one yes/no choice. Our watchman's questions are *imperfect*, so each one removes only a fraction of a bit, and measuring that fraction is the job of Section 9.

9. Information Gain

After the answer. Once an answer e has arrived, the information it gave us is simply how much uncertainty it removed:

$$IG = H(\text{before}) - H(\text{after}) = H(p) - H\big(P(L \mid e)\big)$$

In Section 5, the watchman's consistent answer took P(L) from 0.5 to 1/3. The entropy fell from 1 to H(1/3) = 0.918 bits, so that answer was worth 0.082 bits.

An answer can also be worth *negative* information. Suppose we are at P(L) = 0.8 (entropy 0.722 bits) and the watchman gives a consistent account of his last round. Then P(L) drops to 0.32 / 0.48 = 0.667 and the entropy *rises* to 0.918 bits. A surprising answer can leave us more confused than before. So IG alone can't be what we optimise: we don't know in advance which answer we'll get.

Before the answer: expected information gain. Section 7 told us what to do. Average the uncertainty after each possible answer, weighted by how likely that answer is, and compare with the uncertainty now:

$$\text{EIG}(Q) = \underbrace{H(L)}_{\text{uncertainty now}} - \underbrace{\sum_{e} P(e)\, H(L \mid e)}_{\text{expected uncertainty after asking } Q}$$

Each piece is something we already know how to compute:

Term	What it is	Where it comes from
H(L)	Entropy of our current belief p	Section 8
e	One possible answer to Q (here: yes or no)	The question's design
P(e)	How likely that answer is, *given what we currently believe*	Law of total probability: P(e) = P(e | L)p + P(e | T)(1 − p)
H(L | e)	Entropy of the belief we would have after answer e	Bayes' theorem, then Section 8

Everything uses only the model and our current belief, so it can be calculated **before** the question is asked. That's the whole point.

Four candidate questions. The watchman is in front of us and P(L) = 0.5. Which should we ask first?

Question	P(yes | L)	P(yes | T)	P(yes)	P(L | yes)	P(L | no)	Expected H after	EIG (bits)
Q1: consistent account of last round	0.40	0.80	0.60	0.333	0.750	0.8755	0.1245
Q3: contradicts his earlier timing	0.50	0.10	0.30	0.833	0.357	0.8532	0.1468
Q6: asks why we want to know	0.55	0.45	0.50	0.550	0.450	0.9928	0.0072
Q7: can describe the monitor layout	0.30	0.90	0.60	0.250	0.875	0.7042	0.2958

For Q7 in full: P(yes) = 0.3 × 0.5 + 0.9 × 0.5 = 0.6. A "yes" gives P(L) = 0.15 / 0.6 = 0.25, with H(0.25) = 0.811. A "no" gives P(L) = 0.35 / 0.4 = 0.875, with H(0.875) = 0.544. Expected entropy after = 0.6 × 0.811 + 0.4 × 0.544 = 0.704 bits. So EIG = 1 − 0.704 = 0.296 bits.

Q7 is the clear winner, worth more than twice as much as Q1. The reason is visible in the likelihoods. Someone genuinely at his post would almost certainly know the monitor layout (0.9), while a liar often can't (0.3), so *both* answers are strongly informative: "no" has likelihood ratio 0.7 / 0.1 = 7. Q6 is almost worthless: liars and truth-tellers behave nearly the same (0.55 vs 0.45), so whatever the answer is, we barely move.

This is the moment the project becomes an **optimisation problem**. We have a set of options (questions), a quantity we can calculate for each (EIG), and we choose the option that maximises it:

$$Q^* = \operatorname{argmax}_Q \text{EIG}(Q)$$

("arg max" means "the Q that gives the largest value", rather than the largest value itself.)

Why EIG is never negative. A single answer can increase uncertainty, but the *expected* gain is always at least zero. This follows from two facts we already have. From Section 7, the possible posteriors always average back to the prior. From Section 8, the entropy curve is a dome. Mark the two possible posteriors on the dome and join them with a straight line. The expected entropy after asking is a point on that chord, directly above the prior, and because the dome bulges upwards, that chord point sits *below* the curve at the prior. So expected entropy after ≤ entropy now. (This is a special case of Jensen's

inequality.) The two are equal only when both answers leave us exactly where we started, as with Q9, "mentions the weather", where P(yes | L) = P(yes | T).

A second way to see the same number. Rearranging the definitions shows that

$$\text{EIG}(Q) = H(\text{answer}) - [\, p\, H(\text{answer} \mid L) + (1-p)\, H(\text{answer} \mid T)\,]$$

That is: how unpredictable the answer is, minus how unpredictable it would *still* be if we knew the truth. For Q7: H(0.6) − [0.5 H(0.3) + 0.5 H(0.9)] = 0.971 − 0.675 = 0.296, the same answer. For Q6 the answer is maximally unpredictable (H = 1), but it would be almost as unpredictable even if we knew the truth, which is why it tells us nothing. This quantity is called the **mutual information** between the truth and the answer. It is symmetric: what the answer tells us about the truth equals what the truth would tell us about the answer. (Our simulation code checks numerically that both formulas agree for every question.)

EIG depends on where we stand. The same question is worth different amounts at different beliefs:

Question	EIG at P(L) = 0.2	EIG at P(L) = 0.5	EIG at P(L) = 0.8
Q1	0.084	0.125	0.078
Q3	0.105	0.147	0.088
Q7	0.209	0.296	0.183

Every question is worth most when we are most uncertain; near certainty there is less uncertainty left to remove. This is why the best question has to be recalculated after every answer, which is what the algorithm in Section 11 will do.

10. The Mathematics of Deduction

So far there were only two possibilities: lie or truth. Real deduction usually starts with many. Suppose the museum has 64 staff with keys, and exactly one took the painting. We may only ask yes/no questions, and (for now) every answer is perfectly reliable.

The classic strategy is to cut the possibilities in half each time: "Is the thief's staff number below 33?"

$$64 \to 32 \to 16 \to 8 \to 4 \to 2 \to 1$$

That takes 6 questions, because $2^6 = 64$. In general, N equally likely possibilities need $\log_2 N$ perfect halving questions.

Now look at this through Section 8. With 64 equally likely suspects, the entropy is

$$H = -\sum_{i=1}^{64} \frac{1}{64} \log_2 \frac{1}{64} = \log_2 64 = 6 \text{ bits}$$

So the "6 questions" and the "6 bits" are the same number, and that's not a coincidence. A perfectly reliable question whose two answers are equally likely removes exactly 1 bit. Deduction is the process of shrinking the space of possibilities; information theory gives us a mathematical way to measure how quickly we shrink it.

Why halving works: the reliable case. If answers are perfectly reliable, knowing the truth tells you the answer for certain, so the second term in the mutual-information form of EIG (Section 9) is zero:

$$\text{EIG} = H(\text{answer}) - \underbrace{H(\text{answer} \mid \text{truth})}_{=0 \text{ for perfect answers}} = H(\text{answer})$$

For a yes/no answer, H(answer) is largest, 1 bit, when yes and no are equally likely. So with perfect answers, the best question is the one whose answer is hardest to predict. With equally likely suspects, that means splitting them into equal-sized groups.

When the possibilities are not equally likely. Suppose there are four suspects, but the evidence so far makes them unequal:

Suspect	A	B	C	D
Probability	0.5	0.25	0.125	0.125

The "split the list in half" question is "Is it A or B?". But "yes" has probability 0.75, so this question is worth only H(0.75) = 0.811 bits. The question "Is it A?" splits the *probability* evenly (0.5 vs 0.5) and is worth a full bit, even though it separates one suspect from three.

Carry this on. Ask "Is it A?"; if not, the remaining probabilities are 0.5, 0.25, 0.25, so ask "Is it B?"; if not, ask "Is it C?". The expected number of questions is

$$0.5 \times 1 + 0.25 \times 2 + 0.25 \times 3 = 1.75$$

The halving strategy always needs 2. And the entropy of the starting distribution is

$$H = 0.5\log_2 2 + 0.25\log_2 4 + 2 \times 0.125\log_2 8 = 0.5 + 0.5 + 0.75 = 1.75 \text{ bits}$$

exactly the same as the expected number of questions. A deep result of information theory (Shannon's source coding theorem) says this is no accident: no yes/no questioning strategy can use fewer than H questions on average, and a good strategy can always get within one question of it. Entropy is the true cost of finding out.

So the rule is not "split the *possibilities* equally". It is "split the *probability* equally", which is the same thing as "maximise the entropy of the answer".

Real questions are messier. Three more complications take us back to the watchman:

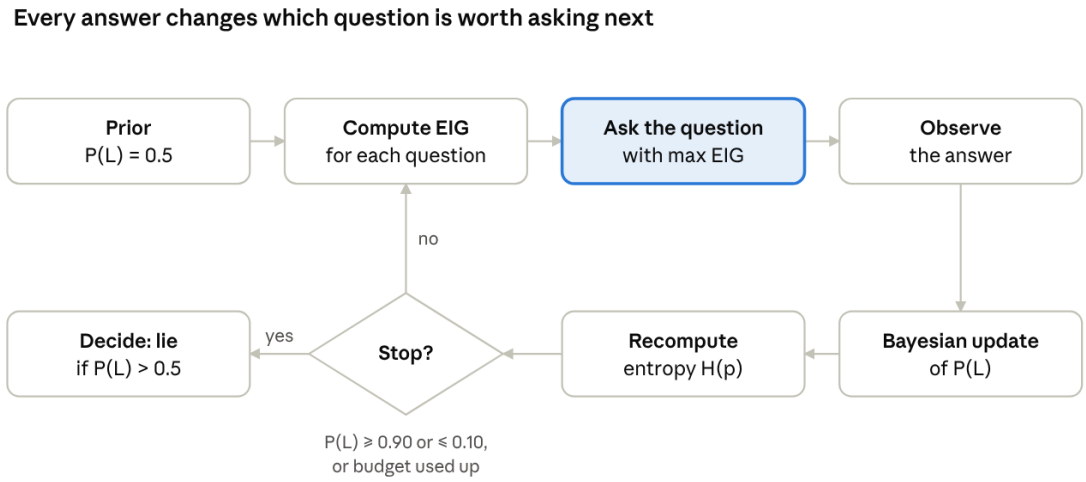
- *Questions can have more than two answers.* A question with three equally likely answers can remove up to $\log_2 3 \approx 1.585$ bits.
- *Answers can be noisy.* Now H(answer | truth) is no longer zero, and balance stops being enough. Q6 ("asks why we want to know") has a perfectly balanced answer, P(yes) = 0.5, yet is worth only 0.007 bits. Q3 is lopsided, P(yes) = 0.3, yet is worth 0.147 bits. Q6's answer is unpredictable, but mostly because of noise, not because of the truth.
- *Some questions are just much more informative than others.* That depends on how differently liars and truth-tellers answer, which is exactly what EIG measures.

This is why the watchman is harder than the 64 suspects. There is only 1 bit of uncertainty to remove (two possibilities, 50/50), but no single question can remove it all. The best question removes 0.296 bits

on average, and the others less. Deduction with noisy evidence is a matter of stacking up fractions of a bit. The best strategy is the one that stacks them fastest, and that is not necessarily the one that splits the possibilities equally. Expected information gain is the general rule that covers every case: perfect or noisy, equal or unequal, two answers or many.

11. Designing the Interrogation Algorithm

We now have every piece. Bayes' theorem updates belief (Section 5). Entropy measures uncertainty (Section 8). Expected information gain scores questions before they are asked (Section 9). Section 10 showed why maximising EIG, and not simply splitting possibilities, is the right general rule. Putting them in a loop gives an algorithm.



the interrogation algorithm · 7 steps, 1 decision, 1 loop

The loop is the important part. After every answer the belief changes, so the EIG of every remaining question changes too, and the next choice has to be recalculated from scratch.

The precise algorithm. The inputs are a prior p_0, a question bank where each question Q has known values a_Q = P(yes | L) and b_Q = P(yes | T), a question budget B, and thresholds (0.10, 0.90).

1. Set p = p_0 and mark every question as unasked.
2. **Check the stopping rule.** If p ≥ 0.90, p ≤ 0.10, B questions have been asked, or no questions remain, go to step 8.
3. For every unasked question Q, compute P(yes) = a_Q p + b_Q(1 − p), the two possible posteriors by Bayes' theorem, and EIG(Q) = H(p) − [P(yes) H(p_yes) + P(no) H(p_no)].
4. Choose Q* with the largest EIG. (If two questions tie, take the first in the list, so the rule is fully determined.)
5. Ask Q* and observe the answer.
6. Update p with Bayes' theorem, using a_Q* and b_Q* if the answer is yes, or 1 − a_Q* and 1 − b_Q* if it is no. Mark Q* as asked.
7. Recompute H(p) and return to step 2.

8. **Decide.** Declare "lie" if $p > 0.5$ and "truth" if $p < 0.5$ (if p is exactly 0.5, the evidence is balanced and we flip a fair coin).

About the stopping rule. The thresholds 0.90 and 0.10 are a modelling choice, not a mathematical fact. They mean "stop once we are 90% confident either way". A court might demand 0.99; a quick screening might accept 0.75. Stricter thresholds give more accurate decisions but need more questions. There is no single "correct" threshold: that depends on how costly a mistake is compared with the cost of another question. Section 14F replaces the fixed threshold with an explicit cost per question.

The algorithm also makes one design decision worth naming. Each question may be asked **only once**. If a question could be repeated with independent answers, the algorithm would simply ask the single best question over and over, and repeating a question is exactly the situation where the independence assumption (A4) is least believable. Section 14C examines repeated, correlated answers directly.

One more honest point: choosing the best question *one step at a time* is called a **greedy** (or myopic) strategy. It is not guaranteed to be the best possible plan over several questions, because a question that is modest now might set up a very informative question later. Finding the truly optimal plan means searching a decision tree of every possible question-and-answer sequence, which grows exponentially. Greedy EIG is the standard practical compromise, and the simulation tests how well it actually does.

12. Computational Experiment

The prediction to test is simple: **choosing questions by maximum EIG should reach a confident, correct conclusion using fewer questions than choosing them at random.** Mathematics told us what *should* happen; a simulation checks whether it does.

The setup. Each simulated interrogation works like this:

1. The computer secretly decides whether the watchman is lying, with probability equal to the base rate (0.5 in the baseline).
2. The interrogator starts with prior $P(L) = 0.5$ and the question bank below.
3. The interrogator chooses questions (by max EIG, or at random), and each answer is generated from the true likelihoods for the hidden truth.
4. After every answer: Bayesian update, then recompute entropy.
5. Stop at $P(L) \geq 0.90$ or ≤ 0.10, or after 10 questions (the whole bank). Then decide.

Each strategy was run for **20,000 trials** per condition, using a fixed random seed so every result can be reproduced exactly.

The question bank. Ten questions, each asked at most once. The likelihoods are modelling choices, designed to range from very useful to useless. (Q1 to Q3 keep the values from Section 6.)

Question	Feature observed ("yes")	P(yes | L)	P(yes | T)	EIG at P(L) = 0.5
Q1	Consistent account of last round	0.40	0.80	0.1245

Question	Feature observed ("yes")	P(yes | L)	P(yes | T)	EIG at P(L) = 0.5
Q2	Hesitates when asked about the alarm	0.60	0.20	0.1245
Q3	Contradicts his earlier timing	0.50	0.10	0.1468
Q4	Consistent account of the keys	0.45	0.75	0.0689
Q5	Gives an unusually precise time	0.35	0.15	0.0393
Q6	Asks why we want to know	0.55	0.45	0.0072
Q7	Can describe the monitor layout	0.30	0.90	0.2958
Q8	Changes a detail when retelling	0.65	0.35	0.0659
Q9	Mentions the weather	0.50	0.50	0.0000
Q10	Names a colleague who can confirm	0.35	0.60	0.0457

Q1 and Q2 have identical EIG. That's not an accident: Q2's "yes" behaves exactly like Q1's "no" (0.6 vs 0.2), so they are the same question with the answers relabelled.

The core code (Python). Every formula is a direct translation of the ones derived in Sections 5 to 9:

```
def entropy(p):
    if p <= 0.0 or p >= 1.0:
        return 0.0                                  # 0 log 0 = 0
    return -p*math.log2(p) - (1-p)*math.log2(1-p)

def bayes_update(p, a, b, answer_yes):
    like_L = a if answer_yes else 1 - a             # P(answer | L)
    like_T = b if answer_yes else 1 - b             # P(answer | T)
    return like_L*p / (like_L*p + like_T*(1 - p))

def expected_info_gain(p, a, b):
    p_yes = a*p + b*(1 - p)                         # total probability
    h_after = (p_yes * entropy(bayes_update(p, a, b, True))
```

```
            + (1 - p_yes) * entropy(bayes_update(p, a, b, False)))
    return entropy(p) - h_after

def choose_eig(p, available, a, b, rng):
    gains = [expected_info_gain(p, a[q], b[q]) for q in available]
    return available[int(np.argmax(gains))]

def choose_random(p, available, a, b, rng):
    return available[rng.integers(len(available))]

STRATEGIES = {"eig": choose_eig, "random": choose_random}

def get_strategy(name):
    key = name.strip().lower()
    if key not in STRATEGIES:
        raise ValueError(f"Unknown strategy {name!r}")
    return STRATEGIES[key]
```

Guarding against the typo bug. An earlier version compared strings, `if strategy == "eig "`, with an accidental trailing space. The comparison was never true, so the "EIG" runs silently fell into the random branch. In this version, a strategy is a *function* passed into the simulation, looked up exactly once through `get_strategy`. An unknown name raises an error instead of quietly doing something else, so the bug cannot happen silently.

Verification. Before any result was recorded, these checks were run, and all passed:

Check	How it was tested	Result
Entropy is correct	H(0.5) = 1, H(0) = H(1) = 0, H(0.9) = H(0.1) = 0.469	Passed
Bayes updates use the observed answer correctly	Reproduced Section 6 exactly: 0.333, 0.600, 0.882, 0.789; the 100-person table (0.5625); a "no" answer uses 1 − a and 1 − b	Passed
EIG is computed correctly	Matched the mutual-information form H(answer) − H(answer | truth) for all 10 questions at P(L) = 0.2, 0.5 and 0.8	Passed
EIG really picks the maximum	At every step of 3,000 trials, asserted the chosen question's EIG ≥ every other available question's EIG	Passed
EIG is deterministic	Its first question was Q7 in all 1,000 trials tested	Passed
Random really is random	First question over 20,000 trials: each of the 10 chosen between	Passed

Check	How it was tested	Result
	1,935 and 2,083 times (2,000 expected)	
Strategy names cannot fall through	"eig" and "random" map to the right functions; a misspelt name raises an error	Passed
Stopping rule works	Every trial that stopped early had P(L) ≥ 0.9 or ≤ 0.1; every other trial used all 10 questions with 0.1 < P(L) < 0.9; no trial passed a threshold without stopping	Passed

The full code (simulation, verification and experiments) is attached separately. Every number in Sections 13 and 14 comes from running it.

13. Results and Analysis

The headline result: maximum-EIG questioning reached the same accuracy as random questioning, but needed far fewer questions: a median of 3 against 7. The prediction from Section 12 held, though not quite in the way you might first expect.

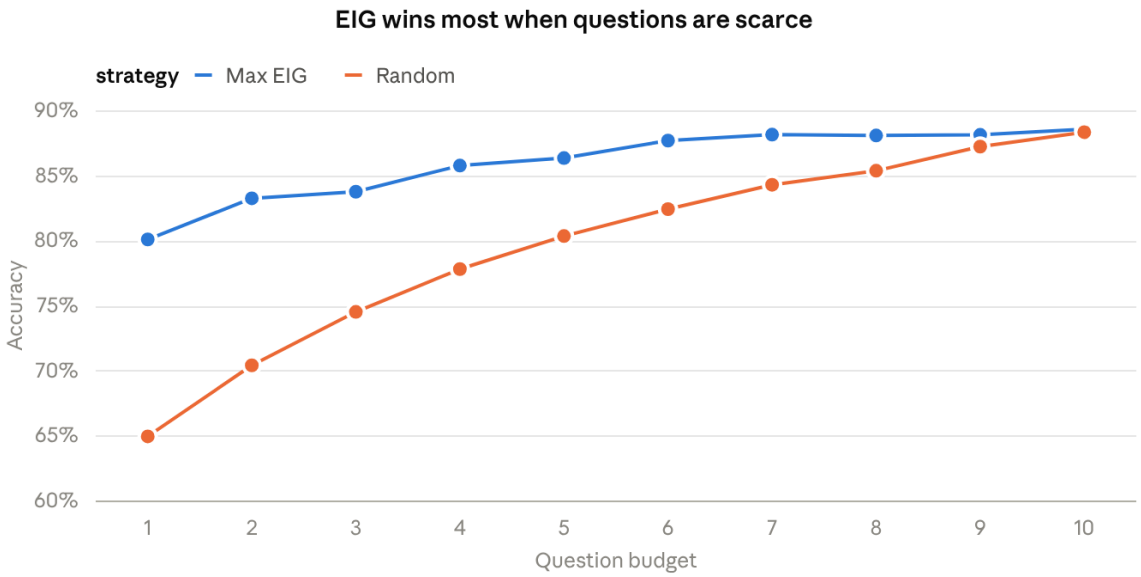
Baseline (prior 0.5, 20,000 trials each)	Max EIG	Random
Accuracy (correct decision)	88.4%	88.1%
Mean questions asked	4.69	6.68
Median questions asked	3	7
Finished within 3 questions	65.2%	14.8%
Reached 90% confidence	78.4%	77.2%
Accuracy when 90% confidence was reached	93.3%	93.1%
Average information per question asked (bits)	0.184	0.110
Average total information gained (bits)	0.544	0.541

Why is final accuracy almost the same? This surprised me at first, but Section 6 explains it. Both strategies have the same ten questions. If the threshold is never reached, both end up asking all ten, and with all ten answers the posterior is *identical* whatever order they were asked in, because likelihood ratios multiply and multiplication doesn't care about order. The strategies can only differ in when they

stop, not in what they finally know. So EIG doesn't make the evidence better. It gets the same evidence faster.

There is also a ceiling. Asking all ten questions every time gives 88.8% accuracy (Section 14F), and neither strategy can beat that, because the question bank itself only contains about that much information about the truth. Both strategies land close to the ceiling.

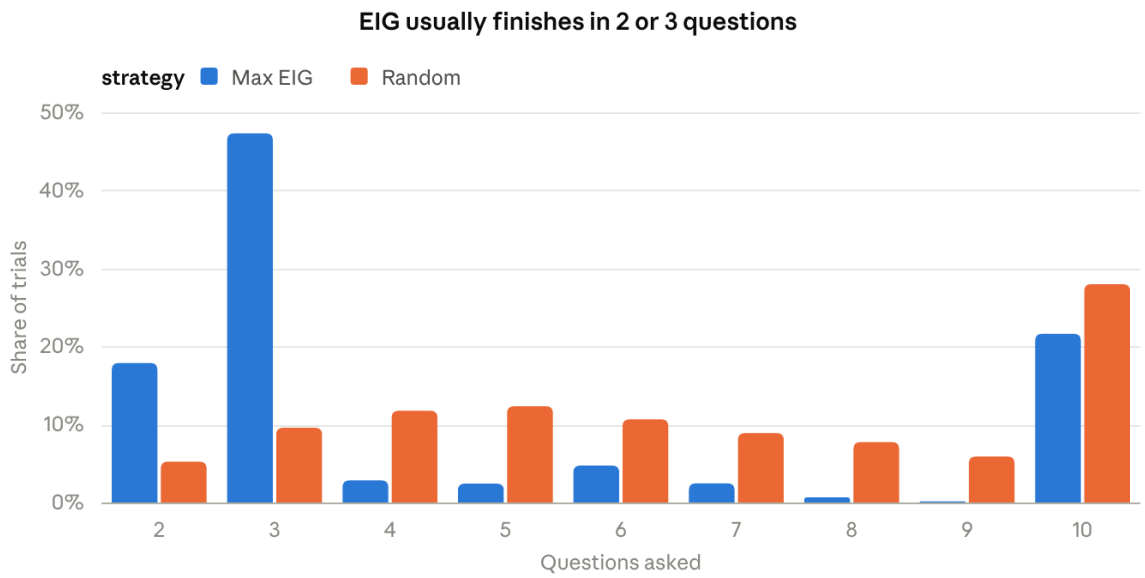
Where EIG wins: when questions are limited. With a fixed budget, the difference is large:



Share of 20,000 simulated interrogations decided correctly, by maximum number of questions allowed (stopping rule 0.10/0.90 still applies).

simulation output · 20,000 trials per budget per strategy

With one question, EIG is right 80.1% of the time against 64.9% for random: it always asks Q7, while random usually asks something weaker. The gap shrinks as the budget grows (83.7% vs 74.5% at 3 questions, 86.3% vs 80.3% at 5) and almost disappears at 10 (88.6% vs 88.3%), because with a big budget even a random order eventually asks the good questions. **Choosing well matters most when questions are scarce.**

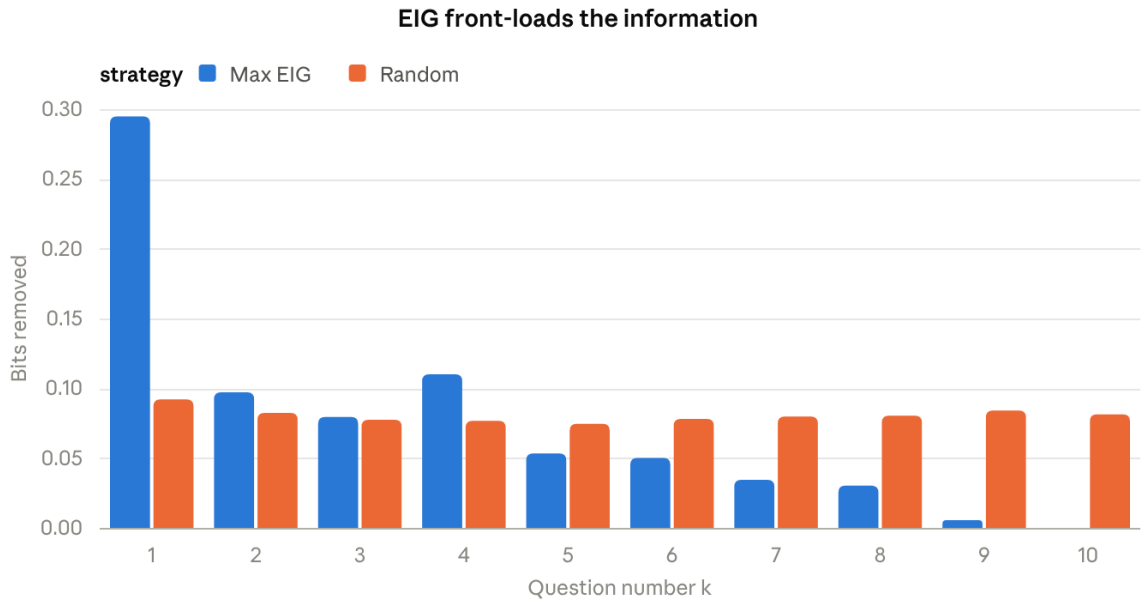


Share of 20,000 trials by number of questions asked. The bar at 10 is mostly trials that used the whole bank without reaching 90% confidence.

simulation output · 20,000 trials per strategy, baseline

EIG never finishes in fewer than 2 questions. That's because no single answer reaches the threshold: even Q7's strongest answer ("no") only reaches P(L) = 0.875. After two questions, for example Q7 "no" and Q3 "yes", the odds are 7 × 5 = 35 and P(L) = 35/36 ≈ 0.97. In practice, EIG ran the same opening every time, Q7 then Q3, and then Q1 in 82% of trials. It asked the useless Q9 ("mentions the weather") only in the 21.6% of trials that used the whole bank. It never wastes a question on noise while anything better is left.

Which questions carried the information?



Average entropy removed by the k-th question, among trials still running at step k.

simulation output · 20,000 trials per strategy, baseline

The first EIG question removed 0.295 bits on average, almost exactly its *predicted* EIG of 0.296. The first random question removed 0.092 bits, and the average EIG of all ten questions at P(L) = 0.5 is 0.0919. So the simulated averages match the theory's expectations. This is a strong sign that the EIG calculation measures something real.

After the first question, EIG's value per question declines: **diminishing returns**, because the best questions are used first. There is one bump, at question 4, and it is worth explaining rather than hiding. The average at step k only includes trials still running. Trials that survive three questions are the ones where answers conflicted, and their beliefs sit much nearer 0.5 (average distance from 0.5 of 0.16, against 0.30 after steps 1 and 2). Section 9 showed that questions are worth most near 0.5, so the survivors get more out of question 4. Random questioning, by contrast, gets roughly the same small amount (about 0.08 bits) from every question, because a random question is an average question.

Honest calibration check. Whenever an interrogation stopped at 90% confidence, the decision was right about 93% of the time for both strategies. It's at least 90%, as Bayes' theorem promises when the model is correct (it's slightly above 90% because a posterior often jumps past the threshold rather than landing exactly on it). Section 14 shows this promise is exactly what breaks when the model is wrong.

14. Extensions

The baseline model worked. Now we break its assumptions one at a time and watch what happens. Every figure below comes from 20,000 simulated trials per condition.

A note on noise. Simulation results have random error of their own. With 20,000 trials and accuracy near 88%, the standard error is about $\sqrt{(0.88 \times 0.12 / 20{,}000)} \approx 0.23$ percentage points. So differences in accuracy smaller than about half a percentage point should be read as "no real difference". (This applies to Section 13 too: 88.4% vs 88.1% is a tie.)

A. Different priors

What if lying is rare, or common? Here the interrogator's prior equals the true base rate of lying.

Prior P(L)	Strategy	Accuracy	Mean questions	Median questions	Accuracy of just guessing the likelier answer
0.2	Max EIG	90.5%	2.32	1	80%
0.2	Random	90.7%	4.43	3	80%
0.5	Max EIG	88.3%	4.71	3	50%
0.5	Random	87.9%	6.72	7	50%
0.8	Max EIG	91.0%	3.47	1	80%
0.8	Random	90.6%	4.60	3	80%

A lopsided prior means less starting uncertainty (H(0.2) = 0.722 bits instead of 1), so fewer questions are needed. At P(L) = 0.2 a single "yes" to Q7 gives P(L) = 0.06 / 0.78 ≈ 0.077, already below 0.10, which is why the median is 1. Higher accuracy here is partly an illusion, though: at P(L) = 0.2 you can be 80% right by always saying "truth" without asking anything. The questioning adds about 10 points at the extremes, against 38 points at 0.5. In every case, EIG kept its speed advantage and chose Q7 first.

A wrong prior. If the interrogator assumes 0.5 when the true rate is 0.2, accuracy drops from 90.5% to 88.4%. When the true rate is 0.8, it drops from 91.0% to 87.5%. The damage is real but modest, because strong evidence gradually outweighs a wrong starting point.

B. Noisy likelihood estimates

In reality nobody knows P(yes | L) exactly. Here the answers are generated from the true likelihoods, but the interrogator believes values that are off by a random error of up to ±δ (drawn afresh for each trial, and kept between 0.02 and 0.98).

Error δ	EIG accuracy	EIG mean questions	EIG: right when it claimed ≥ 90%	Random accuracy	Random mean questions	Random: right when it claimed ≥ 90%
0	88.2%	4.68	93.1%	88.0%	6.70	92.7%
0.1	86.4%	4.22	90.2%	87.1%	6.44	91.3%
0.2	83.1%	3.72	85.9%	83.4%	5.95	86.9%
0.3	79.1%	3.06	81.0%	79.5%	5.33	81.4%

Three things happen, and one is counter-intuitive.

1. **Accuracy falls**, as expected: wrong likelihoods mean wrong updates.
2. **The interrogator stops sooner.** Random errors on average *widen* the believed gap between P(yes | L) and P(yes | T), so the interrogator believes its questions are stronger than they are. Belief moves too far per answer and hits the threshold early.
3. **Confidence stops meaning what it says.** At δ = 0.3 the interrogator claims at least 90% confidence but is right only about 81% of the time. The Bayes guarantee from Section 13 only holds when the model is correct.

The EIG strategy also loses its (tiny) accuracy edge. At δ = 0.1, random is ahead by 0.7 points. A likely reason is sometimes called the **optimiser's curse**: when you pick the question with the *highest estimated* EIG, you tend to pick one whose estimate happens to be too high. So the questions you trust most are, on average, the ones you over-trust. EIG still finishes much faster (3.06 vs 5.33 questions at δ = 0.3), but maximising a quantity is only as good as your estimate of it.

C. Correlated evidence

Section 6 multiplied likelihood ratios, which assumes answers are conditionally independent given the truth (A4). Suppose that fails: the same hesitation question is asked repeatedly, and a person who

hesitated once tends to hesitate again, for reasons that have nothing to do with lying (nerves, habit, a stammer). Multiplying then counts the same clue many times.

To see how much this matters, we need *some* model of the dependence. The one below is an **illustrative modelling assumption**, chosen because it is simple, not a general law of how answers correlate:

> *Each repeat of the question (P(yes | L) = 0.6, P(yes | T) = 0.2) simply **copies the first answer** with probability ρ; otherwise it is a fresh, independent answer.*

Every single answer still has the same probabilities as before; only the relationship between answers changes. The *naive* interrogator ignores ρ and multiplies as in Section 6. The *model-aware* interrogator uses the correct likelihood under this particular copying model. By the law of total probability over "copied" vs "fresh":

$$P(a_i \mid a_1, L) = \rho \cdot [\, a_i = a_1 \,] + (1 - \rho)\, P(a_i \mid L)$$

where [a_i = a_1] is 1 if the answers match and 0 otherwise (and likewise for T). This formula is exact for this copying model only. It is not a general fix for correlation; different dependence would need a different formula.

Correlation ρ	Naive accuracy	Naive average claimed confidence	Naive mean questions	Aware accuracy	Aware average claimed confidence	Aware mean questions
0	90.6%	90.4%	5.92	90.6%	90.4%	5.92
0.5	74.4%	92.6%	4.82	81.1%	81.1%	8.42
0.8	70.3%	94.4%	4.02	75.0%	74.7%	9.97

The naive interrogator becomes **confidently wrong**: at ρ = 0.8 it claims 94% confidence on average and is right 70% of the time. Treating copies as new evidence makes belief race to the threshold on what is really one clue. The model-aware interrogator isn't magic. With heavily correlated answers there simply isn't much information available, so it asks nearly all ten questions and still only reaches 75%. But its stated confidence matches its actual accuracy almost exactly. **Correct modelling buys honesty, even when it can't buy accuracy.**

D. Limited question budget

From the budget experiment in Section 13:

Budget	EIG accuracy	Random accuracy	Gap	EIG questions actually used	Random questions actually used
3	83.7%	74.5%	9.2 points	2.82	2.95
5	86.3%	80.3%	6.0 points	3.50	4.54

Budget	EIG accuracy	Random accuracy	Gap	EIG questions actually used	Random questions actually used
10	88.6%	88.3%	0.3 points (a tie)	4.70	6.70

The tighter the budget, the more the choice of question matters. With 10 questions, strategy barely affects accuracy, only speed.

E. The adversarial liar

So far the liar just answers according to fixed probabilities. Now suppose he **knows the strategy**. Max-EIG is deterministic, so he knows exactly which questions come first: Q7, Q3, Q2 (the three with the highest starting EIG). He rehearses, and on those three questions he answers exactly like a truthful person would: P(yes | L) becomes equal to P(yes | T).

This changes the mathematics at a basic level. The likelihoods are no longer fixed facts about the world; an opponent is *choosing* them in response to us. That turns the problem from pure probability into a **game** between two players, which is the subject of game theory.

Interrogator	Strategy	Overall accuracy	Liars caught	Truth-tellers cleared	Mean questions	Right when it claimed ≥ 90%
Unaware of the rehearsal	Max EIG	64.8%	38.7%	89.6%	5.82	67.8%
Unaware of the rehearsal	Random	64.2%	39.6%	88.5%	7.62	70.9%
Aware (treats Q7, Q3, Q2 as uninformative)	Max EIG	76.2%	73.6%	78.9%	8.19	91.5%
Aware (treats Q7, Q3, Q2 as uninformative)	Random	76.6%	73.1%	80.1%	9.38	92.5%

The unaware interrogator is badly fooled. The rehearsed answers look innocent, and the interrogator trusts them fully, so over 60% of liars are cleared, often with high claimed confidence. The damage comes from the *wrong model*, not the selection rule: random questioning suffers just as much, because it asks the same questions eventually.

The aware interrogator knows that Q7, Q3 and Q2 now have P(yes | L) = P(yes | T), so their EIG is exactly 0 (Section 9) and it skips them. It must build its case from weak questions: it needs about 8 questions, reaches 90% confidence in only 29% of trials, and ends at 76% accuracy. The liar's real power, in mathematical terms, is to **shrink the gap between P(e | L) and P(e | T)**, because that gap is where all information comes from.

This also reveals a weakness of a *predictable* strategy: its predictability tells the liar what to rehearse. A natural defence from game theory is to randomise among good questions so the opponent can't prepare for all of them. We did not test this; it's a direction for further work.

F. Optimal stopping: when is another question not worth it?

The 90% threshold was a modelling choice. A more principled rule gives each question a **cost** c, measured in the same unit as the benefit (bits). The expected reduction in uncertainty from asking one more question is the best EIG still available:

$$\text{benefit of one more question} = \max_{Q \text{ unasked}} \text{EIG}(Q) = H(p) - \min_{Q} \sum_{e} P(e)\, H(L \mid e)$$

The rule is: **ask another question only if that benefit is at least c**; otherwise stop and decide. (The 0.10/0.90 thresholds are switched off here, so the cost alone decides.)

Cost per question c (bits)	Accuracy	Mean questions	Median questions
0 (ask everything)	88.8%	10.00	10
0.01	88.6%	6.15	6
0.03	88.7%	4.89	4
0.05	87.4%	3.48	3
0.10	84.6%	1.79	2
0.20	80.0%	1.00	1

At c = 0.03 bits, the interrogator matches the accuracy of asking every question (88.7% vs 88.8%, a tie) with fewer than half the questions. Past that, accuracy starts to fall. The last few questions in the bank are worth almost nothing. Knowing when to stop is part of asking well.

Two honest caveats. First, this rule is again **myopic**: it only compares the next question with its cost. Occasionally a question worth less than c could open up a later question worth much more, and a truly optimal rule would have to look ahead through the whole tree of future answers (the technique for this is called dynamic programming). Second, in real life, costs aren't measured in bits. They're measured in time, money or stress, and the two possible mistakes (accusing an honest person, or clearing a liar) usually have very different costs. A fuller model would minimise the *expected cost of errors plus questions*, rather than uncertainty alone.

15. Limitations

This project does not show that mathematics can detect real human lies. It shows what is optimal *inside a model*, and the model rests on assumptions that real people do not follow.

What the model assumes	Why real life is different
The likelihoods P(e | L) and P(e | T) are known	Every number in the question bank was invented. Measuring such probabilities for real people would need careful experiments, and they would vary between people and situations. Section 14B showed that even modest errors make the model overconfident.
Only two hypotheses, lie or truth	Real statements can be partly true, mistaken, exaggerated, or misremembered. An honest person can be wrong, and a liar can say true things.
Answers are random draws from fixed probabilities	Real answers depend on stress, culture, memory, how the question is phrased, and the relationship with the questioner. Nervousness is not the same as deception.
Answers are conditionally independent (baseline)	Real answers are linked by personality and mood. Section 14C showed that ignoring this produces confident wrong conclusions, and we only examined one simple form of dependence.
People understand the questions as intended	Misunderstanding a question produces "evidence" that has nothing to do with lying.
The suspect does not adapt (baseline)	Real liars prepare, and Section 14E showed a prepared liar can defeat a model that ignores preparation.
A fixed, known set of questions	Real interviewers invent questions as they go, and follow-ups depend on content, not just on yes/no features.

There are also limits inside the mathematics itself. The strategy is greedy, choosing one question at a time rather than planning a whole sequence (Section 11). The stopping thresholds and costs were modelling choices, not derived facts. And the simulation measures performance against a world that follows the model's rules exactly; it cannot tell us how the method would perform against reality.

None of this makes the mathematics wrong. Given its assumptions, every result in this project follows logically. But a model is a map, not the territory, and the conclusions are exactly as trustworthy as the assumptions they rest on. What mathematics *can* guarantee is honest reasoning: if you believe these numbers, this is how your beliefs should change, and this is the question you should ask next.

16. Conclusion

The project began with a watchman, a missing painting, and a question: *given uncertainty about whether a statement is truthful, can Bayesian inference and information theory be combined to design a sequence of questions that maximises the probability of identifying the truth with the fewest questions?*

Inside our model, the answer is yes, with some important qualifications.

Bayesian inference tells us how evidence should change belief. Starting from conditional probability, Bayes' theorem converts what the model knows, how liars and truth-tellers behave, into what the investigator needs: the probability that *this* statement is a lie. In odds form, each answer simply multiplies the odds by its likelihood ratio.

Entropy tells us how uncertain we are. It is the average surprise we expect when the truth is revealed: 1 bit at a 50/50 belief, zero at certainty.

Expected information gain tells us which question is most valuable before we know the answer. It turns question choice into an optimisation problem, and it explains why a question is worth something only when liars and truth-tellers answer it differently. The mathematics of deduction then showed that this is the general form of the familiar halving strategy: split the probability, not the list of possibilities.

Combining these gives a sequential questioning strategy: update, measure, choose the best next question, repeat, and stop when confident enough or when another question isn't worth its cost.

The simulation tested whether that strategy works. Across 20,000 trials, maximum-EIG questioning reached the same accuracy as random questioning (about 88%, close to the limit of what the question bank contains) but needed a median of 3 questions instead of 7. With only 3 questions allowed, it was right 83.7% of the time against 74.5%. The average information from its first question matched the theory's prediction almost exactly. So the answer to the research question is: the strategy does not identify the truth *more* often than asking everything, but it identifies it with far fewer questions, and when questions are scarce, it identifies it more often.

The extensions showed where this stops. When the likelihoods were wrong, when answers were correlated, or when the liar rehearsed, the mathematics still ran, but its confidence stopped being trustworthy. Every guarantee depended on the model being right.

So the most accurate summary of what this project found is this:

> ***The mathematics does not detect lies. It detects uncertainty and reduces it.***

It tells us how much we don't know, which question will teach us the most, and when we have learned enough. Whether the watchman lied is still, in the end, a question about a human being. What mathematics can offer is the discipline to reason about that question honestly and efficiently.

Appendix A. Simulation code

The complete model used for every result in Sections 12 to 14 (`deception.py`). The experiment runner (`experiments.py`) and verification script (`verify.py`) are supplied as separate files; running `python verify.py` and then `python experiments.py` reproduces every number in this project exactly, because all random seeds are fixed.

```
import math
import numpy as np

# The question bank: name, a = P(yes | Lie), b = P(yes | Truth)
QUESTIONS = [
    ("Q1 consistent account of last round",     0.40, 0.80),
    ("Q2 hesitates when asked about the alarm", 0.60, 0.20),
    ("Q3 contradicts his earlier timing",       0.50, 0.10),
    ("Q4 consistent account of the keys",       0.45, 0.75),
    ("Q5 gives an unusually precise time",      0.35, 0.15),
    ("Q6 asks why we want to know",             0.55, 0.45),
    ("Q7 can describe the monitor layout",      0.30, 0.90),
    ("Q8 changes a detail when retelling",      0.65, 0.35),
    ("Q9 mentions the weather",                 0.50, 0.50),
    ("Q10 names a colleague who can confirm",   0.35, 0.60),
]
NAMES  = [q[0] for q in QUESTIONS]
A_TRUE = np.array([q[1] for q in QUESTIONS])
B_TRUE = np.array([q[2] for q in QUESTIONS])
N_Q = len(QUESTIONS)

def entropy(p):
    """Binary entropy H(p) in bits, with 0 log 0 = 0 (Section 8)."""
    if p <= 0.0 or p >= 1.0:
        return 0.0
    return -p * math.log2(p) - (1 - p) * math.log2(1 - p)

def bayes_update(p, a, b, answer_yes):
    """Posterior P(L | answer) by Bayes' theorem (Section 5)."""
    like_L = a if answer_yes else 1 - a
    like_T = b if answer_yes else 1 - b
    return like_L * p / (like_L * p + like_T * (1 - p))

def expected_info_gain(p, a, b):
    """EIG = H(L) - sum_e P(e) H(L | e) (Section 9)."""
    p_yes = a * p + b * (1 - p)
    p_no = 1 - p_yes
    h_after = 0.0
    if p_yes > 0:
        h_after += p_yes * entropy(bayes_update(p, a, b, True))
    if p_no > 0:
        h_after += p_no * entropy(bayes_update(p, a, b, False))
    return entropy(p) - h_after

# Strategies are functions, so a string typo cannot select the wrong one
def choose_eig(p, available, a_bel, b_bel, rng):
    gains = [expected_info_gain(p, a_bel[q], b_bel[q]) for q in available]
    return available[int(np.argmax(gains))]      # ties -> first in list

def choose_random(p, available, a_bel, b_bel, rng):
    return available[rng.integers(len(available))]

STRATEGIES = {"eig": choose_eig, "random": choose_random}

def get_strategy(name):
```

```
    key = name.strip().lower()
    if key not in STRATEGIES:
        raise ValueError(f"Unknown strategy {name!r}")
    return STRATEGIES[key]

def run_trial(rng, strategy, prior=0.5, base_rate=None, budget=10,
              upper=0.90, lower=0.10, a_true=A_TRUE, b_true=B_TRUE,
              a_bel=None, b_bel=None, cost=None, check_eig=False):
    """One interrogation (the algorithm of Section 11).
    a_true/b_true: how answers are really generated.
    a_bel/b_bel:   what the interrogator believes (default: the truth)."""
    if base_rate is None:
        base_rate = prior
    if a_bel is None:
        a_bel, b_bel = a_true, b_true
    is_lie = rng.random() < base_rate           # the hidden truth
    p = prior
    available = list(range(N_Q))
    asked, entropies = [], [entropy(p)]
    stopped_by = "budget"
    while len(asked) < budget and available:
        if p >= upper or p <= lower:            # stopping rule
            stopped_by = "threshold"
            break
        if cost is not None:                    # Section 14F
            best = max(expected_info_gain(p, a_bel[q], b_bel[q]) for q in available)
            if best < cost:
                stopped_by = "cost"
                break
        q = strategy(p, available, a_bel, b_bel, rng)
        if check_eig:                           # verification only
            gains = {k: expected_info_gain(p, a_bel[k], b_bel[k]) for k in available}
            assert gains[q] >= max(gains.values()) - 1e-12
        prob_yes = a_true[q] if is_lie else b_true[q]
        answer_yes = rng.random() < prob_yes    # the watchman answers
        p = bayes_update(p, a_bel[q], b_bel[q], answer_yes)
        available.remove(q)
        asked.append(q)
        entropies.append(entropy(p))
    else:
        if p >= upper or p <= lower:
            stopped_by = "threshold"
    if p > 0.5:
        says_lie = True
    elif p < 0.5:
        says_lie = False
    else:
        says_lie = rng.random() < 0.5           # balanced evidence
    return {"is_lie": is_lie, "p_final": p, "n": len(asked), "asked": asked,
            "entropies": entropies, "correct": says_lie == is_lie,
            "stopped_by": stopped_by}
```

Appendix B. Verification output

Output of `python verify.py`, exactly as printed when the project's results were generated:

```
entropy ok: 1.0 0.469 0.8813 0.0808
bayes ok: section 6 chain = 0.3333333333333333 0.6 0.8824 0.7895
EIG == mutual information ok
strategy lookup ok
EIG picks max in 3000 trials ok
random first-question counts: [1976, 1998, 2044, 1998, 2033, 1974, 1995, 1964, 1935,
2083]
EIG first question set: ['Q7 can describe the monitor layout']
stopping rule ok
no missed crossings ok
ALL CHECKS PASSED
```